

DPI343

Computer Organization and Assembly Language

Lecture 4.1

Lectures - prof. em. U. Sukovskis

Labs for local students - assoc. prof. P. Rusakovs

Labs for foreign students - assoc. prof. G. Alksnis

Autumn 2025



RTU
DATORZINĀTNES,
INFORMĀCIJAS
TEHNOLOĢIJAS
UN ENERĢĒTIKAS
FAKULTĀTE

Calling library functions from Assembly code

We've already used a number of external function calls in our examples:

Hello.asm (see 1st lecture)

; ...

extern C puts:PROC

extern ExitProcess@4:PROC

.code

start proc

push offset message

call puts

add esp, 4

push 0

call ExitProcess@4

start endp

Directives **extern** declare external names to resolve when linker builds executable code.
Function **puts** is traditional C function from C runtime library (string output to console).
Function **ExitProcess** is Windows API function from kernel32 library (terminates the current process).

You can pass information to a linker to resolve external names in several ways:
by using the directive `includelib`, or by using linker command line parameters
(see, for example, script `bc.bat`).

Parameter value pushed onto top of the stack

Transfer control to the entry point of the function

Unexplained activity so far...

Parameter value pushed onto top of the stack

Transfer control to the entry point of the function

Using Assembly directives to simplify procedure calls

PROTO directive

PROTO directive defines a prototype for a procedure (like a function prototype in C/C++).
Informs the assembler about how many and what types of arguments should be expected in a procedure.

Syntax (simplified):

label **PROTO** [*language-type*] [*parameter*]:*type* [, ...]

label defines name of the procedure. Must exactly match <label> in PROC definition.

Examples:

ExitProcess	PROTO	dwExitCode:DWORD
puts	PROTO	C :DWORD
_ltoa	PROTO	C dwValue:DWORD, lpBuffer:DWORD, dwBase:DWORD

Using Assembly directives to simplify procedure calls

INVOKE directive

INVOKE directive generates executable Assembly instructions to call the function according to its prototype:

INVOKE name [,arguments]

Examples:

```
1) ExitProcess    PROTO    dwExitCode:DWORD
; ...
INVOKE    ExitProcess, 0
```

```
2) puts          PROTO    C :DWORD
; ...
INVOKE    puts, offset Msg1
```

Instead of using INVOKE directive you may use Assembly instructions PUSH to place arguments into stack (in right to left order!) and instruction CALL to transfer control to the procedure:

```
push    offset Msg1
call    puts
add     esp, 4
```

Important!

Proper use of the CALL instruction depends on the calling convention used in the procedure we are calling

More details...

If language type **C** is indicated in the **PROTO** directive then calling convention **cdecl** will be used by the compiler when it generates sequence of CPU instructions to call a procedure. Convention **cdecl** is used by C/C++ compilers. So, if you use C/C++ library functions, you must specify the correct calling convention.

In **cdecl** top of the stack should be adjusted by caller just after procedure returns control to the caller.

To do that we have to increase value of the stack pointer register ESP by the number of bytes pushed into stack as arguments passed to the procedure just after the **CALL** instruction

```
puts          PROTO C:DWORD ; prototype directive
```

```
message byte "Hello!", 0  
; ...
```

Language type C indicated in the prototype

```
invoke puts, offset message ; the directive invoke used to call the puts procedure and pass one argument
```

Compiler will create the following sequence of instructions from this **invoke** directive:

```
push offset message ; 4 bytes item pushed onto stack as an argument passed to the procedure puts  
call puts           ; transfer control to the entry point of the procedure puts  
add esp, 4          ; adjust top of the stack to eliminate the argument value from the top of the stack
```

The other default/standard calling convention **stdcall** is used mostly by Windows API functions. In **stdcall** stack is adjusted by the procedure itself, i.e. by the callee (for example API functions `ExitProcess`, `MessageBox` etc. we've used already).

Useful functions for console output and input

- puts** - Put text string to console
- gets** - Get text string from console input
- gets_s** - Get text string from console with max number of symbols as the 2nd parameter

- scanf** - Gets data from console and stores them according to the parameter format into the locations pointed by the additional arguments
- printf** - Formated data output to console

- atoi** - Converts text string to doubleword binary integer
- _ltoa** - Converts doubleword binary integer to text string

Example: TestIO

Explore and try out the examples **TestIO1.asm** and **TestIO2.asm**.

The first one calls various C functions using the CPU instruction CALL.

The second one does the same, but uses the PROTO and INVOKE directives to simplify the source code for calling functions.

Fragment from TestIO1.asm:

; Use printf to show the result

```
push    res
push    y
push    x
push    offset outformat
call    printf
add     esp, 16
```

The same fragment from TestIO2.asm:

; Use printf to show the result

```
invoke  printf, offset outformat, x, y, res
```

Thank You!

